

The Mathematics of bedrocktrader: Villager Trade Probabilities in Minecraft Bedrock Edition

Rento Saijo*
Connecticut College

2026-07-26

Abstract

Villager trade tables describe random mechanisms rather than a single realized inventory. Turning those tables into probabilities requires more than flattening JSON: source trades are sampled within groups, item choices branch into alternatives, item functions generate additional outcomes, and identical visible offers can arise through several source paths. This note develops the probability model implemented in `bedrocktrader` for the vanilla Minecraft Bedrock Edition 1.26.30.5 trade tables. It formalizes context conditioning and the package’s trade-to-option-to-offer hierarchy, derives selection probabilities when repeated source entries create multiplicity, and shows how exact offers aggregate into item specifications and trade summaries. Separate sections cover librarian enchanted books, capped treasure prices, fully unlocked librarian probabilities, and the documented model used for enchanted equipment. Worked examples recover a 2.79% probability of a fully unlocked librarian offering Mending for at most 26 emeralds and exact Efficiency II diamond-axe probabilities of 6.07% for a toolsmith and 12.13% for a weaponsmith. The model is evaluated analytically and bundled with the package; no simulation, resampling, network access, or runtime JSON parsing is required.

Keywords: Minecraft Bedrock Edition, Villager trading, Enchanting, Probability, R

*Corresponding author: rentosaijo0527@gmail.com.

1 Introduction

The vanilla Minecraft Bedrock Edition trade files look deceptively close to ordinary tabular data. A profession has five tiers, each tier contains one or more groups, and each group contains possible trades. Yet a row in the source does not necessarily describe a row on a player’s trading screen. Groups select subsets of trades, `choice` arrays replace one item with one of several alternatives, contextual filters depend on the villager’s biome variant or dimension, and functions may determine an auxiliary value, map destination, potion, enchantment, or price. Three visibly identical red-candle source entries, for example, represent three chances to obtain the same displayed offer.

bedrocktrader converts those mechanisms into flat base R data frames and associated analysis functions. The package is pinned to Mojang’s immutable Minecraft Bedrock Samples release `v1.26.30.5` (Mojang 2026b). During development, the updater retrieves the vanilla trade tables, villager entity definition, and enchantment registry; verifies each Git blob hash; validates the supported structures; and writes normalized private objects to `R/sysdata.rda`. Installed functions read those bundled objects, so runtime results are offline and reproducible.

This note documents the implemented mathematics. Microsoft specifies the trade-table hierarchy and meanings of `num_to_select`, `wants`, `gives`, `max_uses`, and related fields (Microsoft 2026a,c). Microsoft also describes `enchant_book_for_trading` and `enchant_with_levels`, although the latter delegates important details to the game’s enchanting algorithm (Microsoft 2025). For complete equipment outcomes, bedrocktrader therefore supplements the pinned Mojang files with the enchanting mechanics recorded in a pinned Minecraft Wiki revision (Minecraft Wiki contributors 2026a). The resulting probabilities are exact under that documented model. They should not be read as a claim that every undocumented Bedrock engine constant has been independently verified. Canonical enchantment identifiers and supported level ranges follow Mojang’s pinned registry together with Microsoft’s enchantment documentation (Microsoft 2026b).

The central distinction is resolution. A *trade row* summarizes a flattened source trade and any explicit item-choice combination. An *option row* resolves the concrete item specification while marginalizing over its possible prices. An *offer row* resolves both specification and price. These resolutions answer different questions, and the probability attached to a row always matches what that row displays.

2 Notation and context conditioning

Let $c = (v, d)$ denote the requested villager context, where v is a villager biome variant and d is a dimension. Most profession tables do not use either value. The pinned fisherman table contains variant-dependent boat offers, while the cartographer table uses both variant and dimension filters. bedrocktrader requires the relevant context instead of assigning Plains or the Overworld silently.

For group g , let $\mathcal{S}_g$ be the multiset of authored source entries. A filter indicator

$$I_s(c) = \begin{cases} 1, & \text{when source entry } s \text{ applies under } c, \\ 0, & \text{otherwise} \end{cases}$$

is evaluated before group sizes or probabilities. The applicable multiset is

$$\mathcal{S}_g(c) = \{s \in \mathcal{S}_g : I_s(c) = 1\}, \quad n_g(c) = |\mathcal{S}_g(c)|.$$

This ordering matters. If a group contains several variant-specific alternatives, counting all entries before filtering would dilute the probability of the one alternative that can actually occur for the supplied villager. Package output reports `num_trades = n_g(c)`, counting authored entries rather than the number of visually distinct rows.

Every applicable source path is represented privately even when two paths produce the same public trade. Public identifiers then preserve a nested relationship:

$$\text{trade_id} \longrightarrow \text{option_id} \longrightarrow \text{offer_id}.$$

A `trade_id` identifies one authored trade after identical source entries have been collapsed with their multiplicity retained. Mojang `choice` arrays can create several trade-view rows sharing that identifier. An `option_id` identifies one concrete result specification, such as a complete enchantment set. Its offers share the same specification but may charge different prices. Finally, `offer_id` identifies one exact combination of wants, gives, specification, and price.

3 Source-trade selection

3.1 Selection without multiplicity

Suppose group g has $n = n_g(c)$ applicable authored entries and selects k entries uniformly without replacement. For an ordinary source entry,

$$\Pr(\text{selected} \mid c) = \frac{k}{n}, \quad 0 \leq k \leq n.$$

Mojang's `num_to_select` supplies k . The value `-1` is bedrocktrader's normalized representation of the pinned select-all convention, in which every applicable entry has selection probability one. The pinned fisherman master group is the one known omission: its pufferfish trade and applicable variant-specific boat trade are both selected, so the updater records `num_to_select = -1` and rejects additional unexpected omissions.

3.2 Selection with multiplicity

Several authored entries can yield one public trade. Let w be the number of entries producing that same result. The public row appears if the selected subset contains at least one of those w entries. Of the $\binom{n}{k}$ equally likely subsets, $\binom{n-w}{k}$ avoid all w copies. Consequently,

$$p_{\text{trade}}(n, k, w) = 1 - \frac{\binom{n-w}{k}}{\binom{n}{k}}, \quad (1)$$

where the numerator is zero whenever $n - w < k$. Equation (1) reduces to k/n when $w = 1$.

The librarian master candle group provides a compact example. Mojang authored three identical red-candle entries and one yellow-candle entry, with one of the four selected. Red therefore has

$$1 - \frac{\binom{4-3}{1}}{\binom{4}{1}} = 1 - \frac{1}{4} = \frac{3}{4},$$

whereas yellow has $1/4$. `bedrocktrader` returns one red row and one yellow row because the three red rows would be publicly indistinguishable. It still reports `num_trades = 4`, and the retained private multiplicity supplies $w = 3$.

4 Choices, functions, and row probabilities

A source trade may contain an explicit `choice` array in a want or give position. If position r contains c_r alternatives and the positions are resolved uniformly, one concrete cross-product has conditional probability

$$p_{\text{choice}} = \prod_r \frac{1}{c_r} = \frac{1}{\prod_r c_r}. \quad (2)$$

The source tables can also apply a function to the given item. Let f index a function outcome and $p_{\text{function}}(f)$ be its conditional probability. Uniform auxiliary-value functions assign equal probability to each inclusive value. Fixed potion and exploration-map functions have one resolved outcome with probability one. Enchantment functions use the models developed below. `random_dye` remains unresolved because the pinned static sources do not establish a complete leather-color distribution.

For an exact offer o , the implemented factorization is

$$p_{\text{offer}}(o \mid c) = p_{\text{trade}}(o \mid c) \times p_{\text{choice}}(o \mid c) \times p_{\text{function}}(o \mid c). \quad (3)$$

The factors in (3) describe sequential parts of the same mechanism. Context filtering precedes them, source selection chooses a trade, explicit item choices choose its flat item combination, and the function chooses the generated result.

4.1 Aggregation across resolutions

Let $\mathcal{F}(a)$ be the exact offers associated with option a . The option probability marginalizes over prices:

$$p_{\text{option}}(a \mid c) = \sum_{o \in \mathcal{F}(a)} p_{\text{offer}}(o \mid c). \quad (4)$$

Let $\mathcal{A}(t, b)$ be the options associated with trade identifier t and one explicit base item-choice combination b . The probability shown on that trade-view row is

$$p_{\text{row}}(t, b \mid c) = \sum_{a \in \mathcal{A}(t, b)} p_{\text{option}}(a \mid c). \quad (5)$$

Thus, summing price-specific offers recovers the option row, and summing the options for a base item choice recovers its trade row. Rows across an entire tier need not sum to one: separate groups may each contribute an offer, and a villager can expose several trades simultaneously.

5 Librarian enchanted books

5.1 Enchantment and level

The pinned librarian table contains four independent opportunities for an enchanted book: novice, apprentice, and journeyman book candidates are selected with probability $1/2$, and the expert candidate is selected with probability $1/3$. Conditional on selecting the book trade, the documented model chooses one of $E = 39$ villager-attainable enchantments uniformly ([Minecraft Wiki contributors 2026b](#)). Soul Speed, Swift Sneak, and Wind Burst are recognized by the package registry but excluded from this librarian pool.

Let e be an attainable enchantment and m_e its maximum level. Conditional on e , the level L is uniform on $\{1, \dots, m_e\}$. Therefore,

$$\Pr(E = e, L = \ell \mid \text{book selected}) = \frac{1}{39} \frac{1}{m_e}, \quad \ell = 1, \dots, m_e. \quad (6)$$

Because different enchantments have different maximum levels, enchantment-level pairs are not collectively uniform. Aqua Affinity I has conditional probability $1/39$, while each Efficiency level has conditional probability $1/(39 \cdot 5)$.

5.2 Emerald-price generation

Microsoft gives the pinned `enchant_book_for_trading` parameters as `base_cost = 2`, `base_random_cost = 5`, `per_level_cost = 3`, and `per_level_random_cost = 10` ([Microsoft 2025](#)). For level L , define

$$R_L = 5 + 10L, \quad U_L \sim \text{DiscreteUniform}\{0, \dots, R_L - 1\}.$$

The ordinary price before treasure adjustment is

$$C_0(L) = 2 + 3L + U_L. \quad (7)$$

Hence the normal supports are 5–19, 8–32, 11–45, 14–58, and 17–71 before the universal stack cap. The package applies the 64-emerald cap to final prices, so level V ordinary prices above 64 also merge at 64. For a treasure enchantment,

$$C_T(L) = \min\{2C_0(L), 64\}. \quad (8)$$

The support below 64 is therefore even and can be noncontiguous inside the minimum–maximum interval shown at option resolution.

Let $h_T(x) = \min(2^T x, 64)$, where T is the treasure indicator. Several underlying values can map to the same capped price. For displayed emerald price c ,

$$\Pr(C = c \mid e, L) = \frac{|\{u \in \{0, \dots, R_L - 1\} : h_T(2 + 3L + u) = c\}|}{R_L}. \quad (9)$$

Equation (9) makes the 64-emerald row heavier whenever the cap combines several underlying values. It also prevents a min–max range from implying prices that cannot occur.

Combining (6) and (9), the generator probability for one enchanted-book offer is

$$p_{\text{function}}(e, L, c) = \frac{1}{39m_e} \times \frac{N_{e,L,c}}{5 + 10L}, \quad (10)$$

where $N_{e,L,c}$ is the number of pre-cap random values producing c . Multiplication by the source-trade selection probability completes (3).

5.3 A fully unlocked librarian

Let $j = 1, \dots, 4$ index the four book opportunities, and let A be any query-defined set of acceptable enchantment, level, and price outcomes. The qualifying probability for opportunity j is

$$q_j(A) = s_j \times \sum_{(e,L,c) \in A} \frac{1}{39m_e} \times \frac{N_{e,L,c}}{5 + 10L}, \quad (11)$$

with $s = (1/2, 1/2, 1/2, 1/3)$. Treating the four group selections as independent, the probability that a fully unlocked librarian offers at least one qualifying book is

$$\Pr(\text{at least one qualifying book}) = 1 - \prod_{j=1}^4 \{1 - q_j(A)\}. \quad (12)$$

This complement form avoids double counting librarians whose inventories contain two or more qualifying books.

6 Enchanted equipment

Armorer, fisherman, fletcher, toolsmith, and weaponsmith tables contain 16 `enchant_with_levels` trades. Microsoft describes the function as applying the enchanting-table algorithm with an input level range (Microsoft 2025). In the pinned trades, treasure enchantments are disabled and the source level is uniform from 5 through 19. bedrocktrader enumerates the complete distribution analytically rather than simulating it.

6.1 Source level, price, and base power

Let

$$S \sim \text{DiscreteUniform}\{5, \dots, 19\}. \quad (13)$$

If the authored base emerald cost is b , the exact displayed equipment price is

$$C = b + S. \quad (14)$$

This dependence is important: price and enchantment set share the same source level. Aggregating enchantment sets first and attaching an independent price range would destroy that dependence and give incorrect probabilities under an emerald cutoff.

Let A_i be the item's enchantability and $r_i = \lfloor A_i/4 \rfloor$. Two independent rolls satisfy

$$X_1, X_2 \stackrel{\text{iid}}{\sim} \text{DiscreteUniform}\{0, \dots, r_i\}. \quad (15)$$

The base enchanting power is

$$B = S + 1 + X_1 + X_2. \quad (16)$$

Next, let M follow the symmetric triangular distribution on $[0.85, 1.15]$ with mode 1. Its cumulative distribution function is

$$F_M(m) = \begin{cases} 0, & m \leq 0.85, \\ \frac{(m - 0.85)^2}{2(0.15)^2}, & 0.85 < m \leq 1, \\ 1 - \frac{(1.15 - m)^2}{2(0.15)^2}, & 1 < m < 1.15, \\ 1, & m \geq 1.15. \end{cases} \quad (17)$$

The integer modified power is

$$P = \left\lfloor BM + \frac{1}{2} \right\rfloor. \quad (18)$$

For fixed B , its probability mass follows directly from the CDF:

$$\Pr(P = p \mid B) = F_M\left(\frac{p + 1/2}{B}\right) - F_M\left(\frac{p - 1/2}{B}\right). \quad (19)$$

The updater averages (19) over the 15 source levels and the $(r_i + 1)^2$ equally likely enchantability-roll pairs. This analytic integration preserves small probabilities that a finite simulation could miss.

6.2 Eligible levels and weighted selection

Each enchantment e has a weight w_e , item-compatibility rule, and power interval $[a_{e,\ell}, b_{e,\ell}]$ for level ℓ . At power P , the eligible level is the highest ℓ satisfying

$$a_{e,\ell} \leq P \leq b_{e,\ell}. \quad (20)$$

If no level satisfies (20), the enchantment is absent from the candidate set. Treasure enchantments are removed because every pinned equipment function sets `treasure = false`. The first enchantment is drawn from the eligible set $\mathcal{E}(P)$ with

$$\Pr(e \text{ selected first} \mid P) = \frac{w_e}{\sum_{a \in \mathcal{E}(P)} w_a}. \quad (21)$$

After an enchantment is selected, it and all conflicting enchantments are removed. The implemented conflict families are the four protection enchantments, the three damage enchantments, Fortune versus Silk Touch, and Multishot versus Piercing. Accordingly, the package never returns complete sets containing both members of those incompatible families.

6.3 Additional enchantments

At a recursive step with current power P_r , the probability of attempting another enchantment is

$$\rho(P_r) = \min \left\{ \frac{P_r + 1}{50}, 1 \right\}. \quad (22)$$

If the process continues, the power becomes

$$P_{r+1} = \left\lfloor \frac{P_r}{2} \right\rfloor, \quad (23)$$

and one of the remaining compatible candidates is selected by its relative weight. If none remain, the current set is final. Otherwise, termination at that step has probability $1 - \rho(P_r)$.

For a particular ordered selection path $\pi = (e_1, \dots, e_m)$, the conditional probability is the product of its relative selection weights, continuation probabilities through step $m - 1$, and the final stopping probability:

$$\Pr(\pi \mid P) = \left[\prod_{r=1}^m \frac{w_{e_r}}{\sum_{a \in \mathcal{E}_r} w_a} \right] \left[\prod_{r=1}^{m-1} \rho(P_r) \right] \tau_m, \quad (24)$$

where $\mathcal{E}_r$ is the compatible candidate set at step r , and $\tau_m = 1 - \rho(P_m)$ unless no candidates remain, in which case $\tau_m = 1$. Different paths can finish with the same unordered complete set. `bedrocktrader` sorts its namespaced `identifier=level` pairs and sums all paths sharing that representation:

$$\Pr(H = h \mid S) = \sum_{B, P, \pi: \text{set}(\pi)=h} \Pr(B \mid S) \Pr(P \mid B) \Pr(\pi \mid P). \quad (25)$$

Together, (13), (14), and (25) define the exact equipment offer:

$$\Pr(H = h, C = c) = \sum_{s=5}^{19} \frac{1}{15} \times \mathbf{1}\{c = b + s\} \times \Pr(H = h \mid S = s). \quad (26)$$

The source-trade and explicit-choice factors in (3) are then applied. Option-view rows sum (26) over c , whereas offer-view rows retain each price-specific term.

7 Query probabilities

7.1 Enchanted books

`enchanted_book_probability()` defines A in (11) using one requested `identifier=level` pair and an inclusive original-price cutoff $C_{\max}$. With `include_higher_level = FALSE`, an outcome qualifies when

$$e = e^*, \quad L = L^*, \quad C \leq C_{\max}.$$

When `include_higher_level = TRUE`, the level requirement becomes $L \geq L^*$. The function then applies (12). The cutoff concerns the modeled original offer and does not simulate curing.

7.2 Enchanted equipment

For equipment, write the requested set as

$$Q = \{(e_1, \ell_1), \dots, (e_r, \ell_r)\}$$

and an observed complete set as H . Under `match = 'exact'`, the identifiers in H must equal those in Q . Under `match = 'contains'`, Q 's identifiers need only be a subset of H . The level comparison is equality by default and becomes $L_H(e) \geq \ell_e$ when `include_higher_level = TRUE`.

Let $\mathcal{Q}$ be the offer rows that satisfy the identifier rule, level rule, item rule, profession rule, and $C \leq C_{\max}$. The returned probability is

$$\Pr(\mathcal{Q}) = \sum_{o \in \mathcal{Q}} p_{\text{offer}}(o \mid c). \quad (27)$$

The events are disjoint exact offers, so direct summation is appropriate. Diamond axes require a profession because both toolsmiths and weaponsmiths sell them. Other supported equipment items imply a profession. Recognized but item-inapplicable enchantments, incompatible requests, and unavailable enchantments define an empty $\mathcal{Q}$ and return zero.

8 Worked examples

8.1 Mending for at most 26 emeralds

Mending is a level-I treasure enchantment. From (7), its 15 equally likely ordinary underlying prices run from 5 through 19. After (8), the displayed support is 10, 12, ..., 38. Nine of those 15 values are at most 26, so the conditional price probability is $9/15 = 3/5$. Within a selected book trade,

$$\Pr(\text{Mending I and } C \leq 26) = \frac{1}{39} \frac{3}{5} = \frac{1}{65}.$$

The four opportunity probabilities are therefore

$$q_1 = q_2 = q_3 = \frac{1}{2} \frac{1}{65} = \frac{1}{130}, \quad q_4 = \frac{1}{3} \frac{1}{65} = \frac{1}{195}.$$

Equation (12) gives

$$1 - \left(1 - \frac{1}{130}\right)^3 \left(1 - \frac{1}{195}\right) = 0.027910633 \dots,$$

or approximately 2.79%. The corresponding R call is:

```
mending_probability <- enchanted_book_probability(  
  enchantment = 'minecraft:mending=1',  
  max_emeralds = 26  
)
```

The often-used 26-emerald threshold is a screening rule for a favorable original librarian price. The package does not claim that every such book will become a one-emerald trade, because curing and subsequent price adjustments are outside this model.

8.2 Exact Efficiency II diamond axes

The next query requires the complete enchantment set to be exactly Efficiency II, permits every original price, and does not count higher Efficiency levels:

```
toolsmith_probability <- enchanted_item_probability(  
  item          = 'axe',  
  enchantments = 'efficiency=2',  
  profession    = 'toolsmith'  
)  
  
weaponsmith_probability <- enchanted_item_probability(  
  item          = 'axe',  
  enchantments = 'efficiency=2',  
  profession    = 'weaponsmith'  
)
```

Analytic enumeration of (13)–(26) yields 0.060656412, or approximately 6.07%, for a fully unlocked toolsmith and 0.121312824, or approximately 12.13%, for a fully unlocked weaponsmith. The conditional enchanting distribution is the same diamond-axe distribution in both tables. Their factor-of-two difference comes from source selection: the toolsmith axe trade is selected with probability 1/2, while the weaponsmith axe trade is guaranteed.

Changing `match = 'contains'` would also admit axes that carry Efficiency II alongside compatible enchantments. Setting `include_higher_level = TRUE` would admit Efficiency III and above as well.

8.3 Candle multiplicity and probability conservation

The candle group demonstrates the entire hierarchy without an enchanting function. At trade resolution, three red source entries collapse into one row with probability 3/4; yellow has probability 1/4. Each color is already a concrete specification and fixed price, so its option and offer probability remain unchanged:

$$p_{\text{trade}}(\text{red}) = p_{\text{option}}(\text{red}) = p_{\text{offer}}(\text{red}) = \frac{3}{4}.$$

For enchanted equipment or books, one option contains several prices. Equation (4) becomes an observable conservation check:

$$\sum_{o: \text{option_id}(o)=a} p_{\text{offer}}(o) = p_{\text{option}}(a).$$

Summing those options according to (5) recovers the corresponding trade-view probability. The package’s lightweight tests exercise these identities for all armorer views.

9 Model boundaries

The model deliberately separates source-resolved quantities from documented reconstructions and unresolved engine behavior. `probability_status = 'exact'` means the result follows directly from the pinned table’s selection, choices, and fixed or uniformly enumerated functions. `documented_model` means the probability is exact under the mathematical model in this note. It does not elevate Minecraft Wiki mechanics to an official Mojang guarantee. `random_dye` remains `partial`; bedrocktrader does not invent a distribution for leather colors when the pinned sources do not establish one.

All emerald cutoffs refer to the original modeled price. Demand, depleted stock, player reputation, curing discounts, and Hero of the Village effects can alter the amount shown in game. Those mechanisms operate after the trade outcome modeled here and are outside the package’s current probability functions. The exposed `price_multiplier` remains useful for identifying trade-table behavior, but it is not used to forecast a later price.

The librarian model assumes the documented uniform enchantment-then-level selection among 39 attainable enchantments. The equipment model excludes treasure enchantments because the pinned `enchant_with_levels` entries do so. It also depends on the documented

enchatability values, eligible power intervals, weights, conflicts, continuation rule, and triangular modifier in the pinned reference revision. A future Bedrock release can change identifiers, trade structures, or engine rules; this package therefore supports only 1.26.30.5 until the model can be reviewed against another complete release.

The updater fails closed when a pinned JSON shape, function form, filter, or checksum differs from what the parser understands. No Mojang JSON is redistributed verbatim. The normalized values remain subject to Mojang’s license notice and the Minecraft End User License Agreement (Mojang 2026a). bedrocktrader’s R code is distributed under GPL-3-or-later.

10 Conclusion

Villager trading combines several modest random mechanisms whose interaction is easy to misread from a flattened table. Conditioning on context determines the applicable source pool; selection without replacement produces source-trade probabilities; multiplicity combines identical authored paths; item choices and functions generate conditional branches; and summation connects exact offers to item options and trade summaries. Librarian books add a uniform-enchantment, uniform-level, and capped-price model, while equipment trades couple a source level to both emerald price and a recursive enchanting algorithm.

The `trade_id -> option_id -> offer_id` hierarchy keeps those resolutions visible. It supports concise trade inspection, exact probability queries, and conservation checks without simulation. Most importantly, each public probability answers the question posed by its row: a summarized trade, a specified item over possible prices, or one exact offer.

References

- Microsoft (2025), ‘Loot tables documentation: Enchanting tables’, <https://learn.microsoft.com/en-us/minecraft/creator/reference/content/loottablereference/examples/loottabledefinitions/enchantingtables?view=minecraft-bedrock-stable>. Updated June 25, 2025.
- Microsoft (2026a), ‘Creating a trade table’, <https://learn.microsoft.com/en-us/minecraft/creator/documents/createtradetable?view=minecraft-bedrock-stable>. Updated March 5, 2026.
- Microsoft (2026b), ‘Introduction to enchantments’, <https://learn.microsoft.com/en-us/minecraft/creator/documents/introtoenchantments?view=minecraft-bedrock-stable>.
- Microsoft (2026c), ‘Trade tables documentation: minecraft:tradegroup’, <https://learn.microsoft.com/en-us/minecraft/creator/reference/content/tradetablereference/examples/tradetablecomponents/tradegroup?view=minecraft-bedrock-stable>. Updated March 5, 2026.
- Minecraft Wiki contributors (2026a), ‘Enchanting table mechanics’, https://minecraft.wiki/w/Enchanting_table_mechanics?oldid=3681507. Revision 3681507.
- Minecraft Wiki contributors (2026b), ‘Trading’, <https://minecraft.wiki/w/Trading>.
- Mojang (2026a), ‘Minecraft bedrock samples license notice’, <https://github.com/Mojang/bedrock-samples/blob/v1.26.30.5/LICENSE.md>. Version 1.26.30.5.
- Mojang (2026b), ‘Minecraft bedrock samples v1.26.30.5’, <https://github.com/Mojang/bedrock-samples/releases/tag/v1.26.30.5>. Released June 16, 2026.